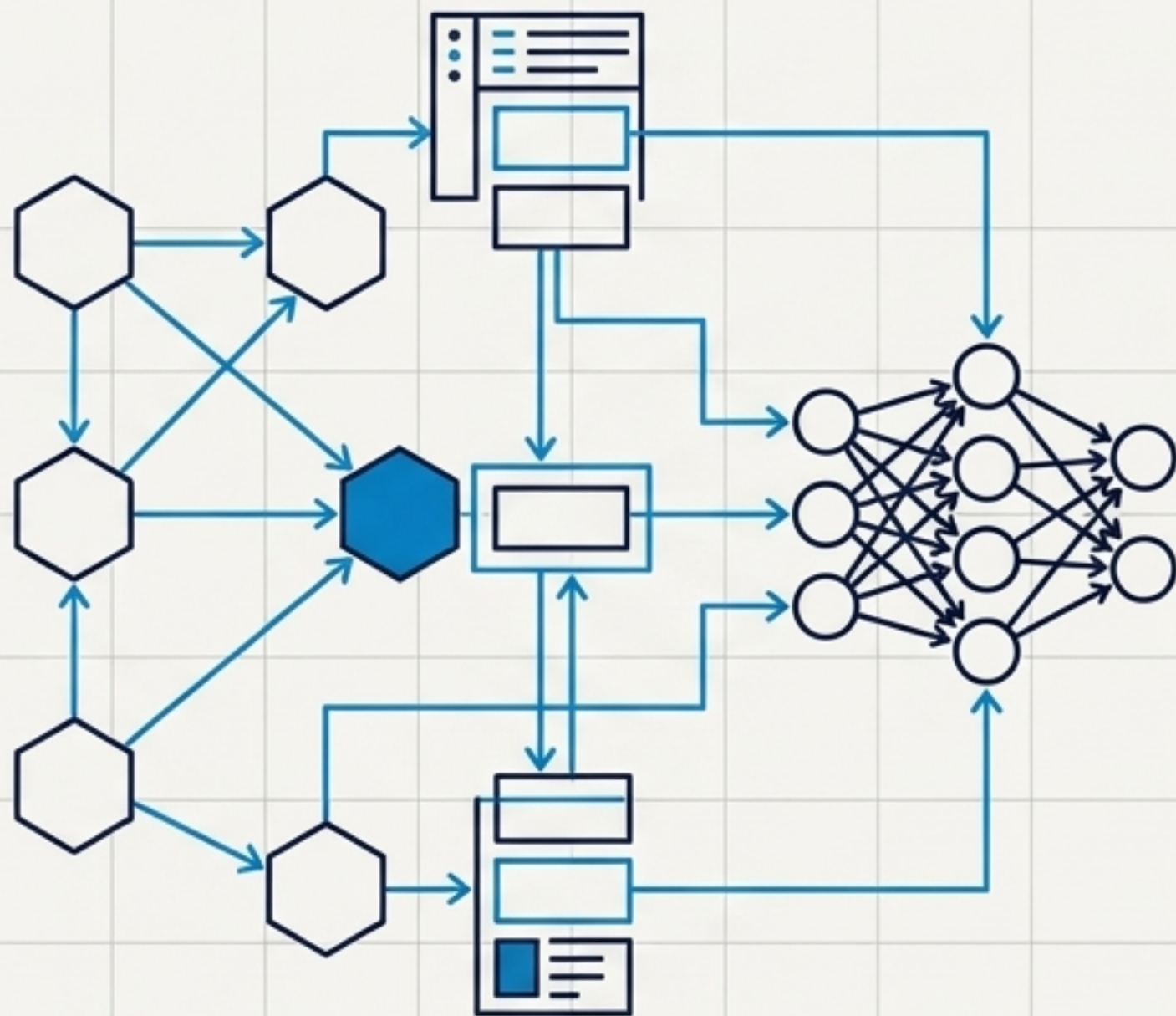




El Playbook del Arquitecto IA

[De Picador de Código a Orquestador de Sistemas]



El Hype y la excusa del AI-Washing tras la sobrecontratación COVID.

Aconsejar a otros que dejen de aprender a escribir código porque la IA lo va a automatizar será recordado como uno de los peores consejos profesionales de la historia. — Andrew Ng

Paradoja del Mercado: Faltan Profesionales Cualificados



El mercado ya no demanda usuarios de chat, sino arquitectos con fundamentos inquebrantables.

La Realidad del Sector en 2026



50%

Developers utilizan IA para programar en su día a día.



30%

Del código generado está escrito por IA.



60%

A favor de las herramientas que utilizan IA en desarrollo.

SWE-Bench Verificado

Evaluación con 500 problemas reales de GitHub validados por humanos.

La IA no es un copiloto opcional, es infraestructura. El humano asume el diseño y la escalabilidad; la IA asume la ejecución manual.

Evolución de Roles: El Coder vs. El Arquitecto

⚠ El Vibe Coder / Obsoleto



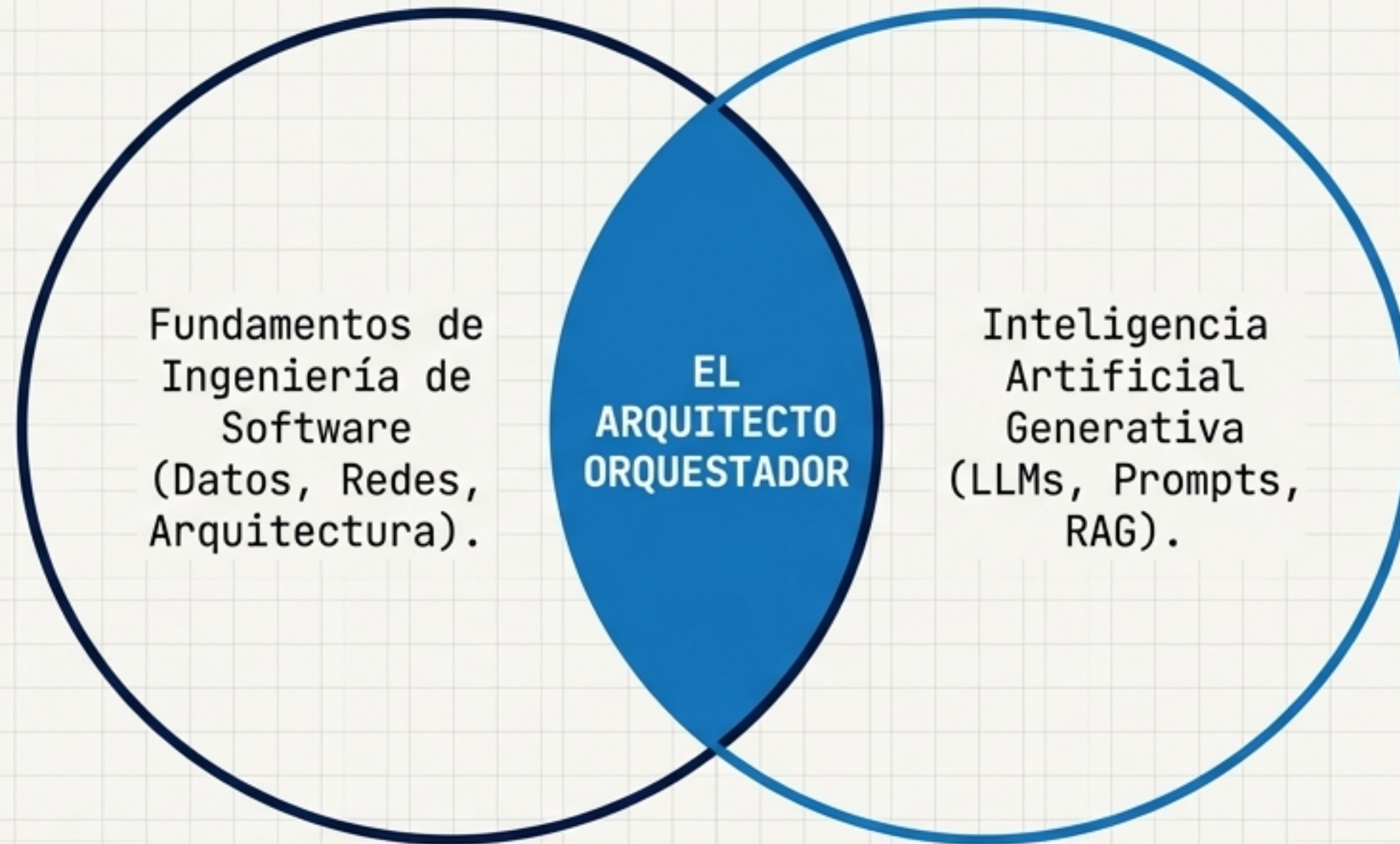
- ⚠ **Foco:** Se centra exclusivamente en la sintaxis.
- ⚠ **Mentalidad:** Teme el impacto de la IA y los despidos.
- ⚠ **Ejecución:** Programa por intuición superficial, sufre con la integración.

✓ El Arquitecto de Software / Moderno



- ✓ **Foco:** Domina los fundamentos de sistemas (APIs, rendimiento, seguridad).
- ✓ **Mentalidad:** Utiliza la IA como motor de ejecución delegada.
- ✓ **Ejecución:** Orquesta agentes, diseña la estructura, asegura la escalabilidad.

La Fusión del Paradigma: Diseñar Sistemas no es Picar Código

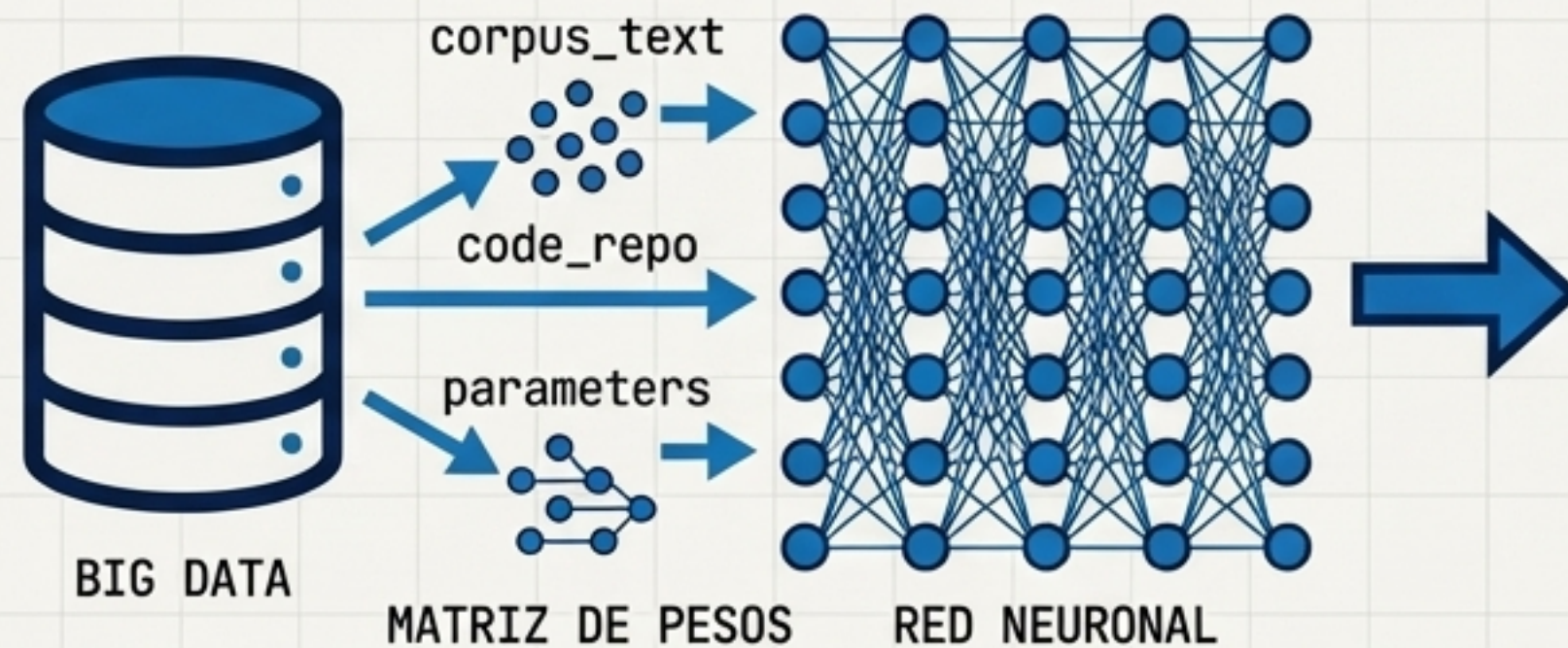


- > No saber programar te da una ventaja, es una completa tontería.
- > Aprende cómo fluyen los datos. Aprende sobre rendimiento. Aprende sobre APIs. Las complejidades de la sintaxis importarán menos, pero cómo encajan las piezas importa más que nunca.

Anatomía de un LLM: El Motor Bajo el Capó

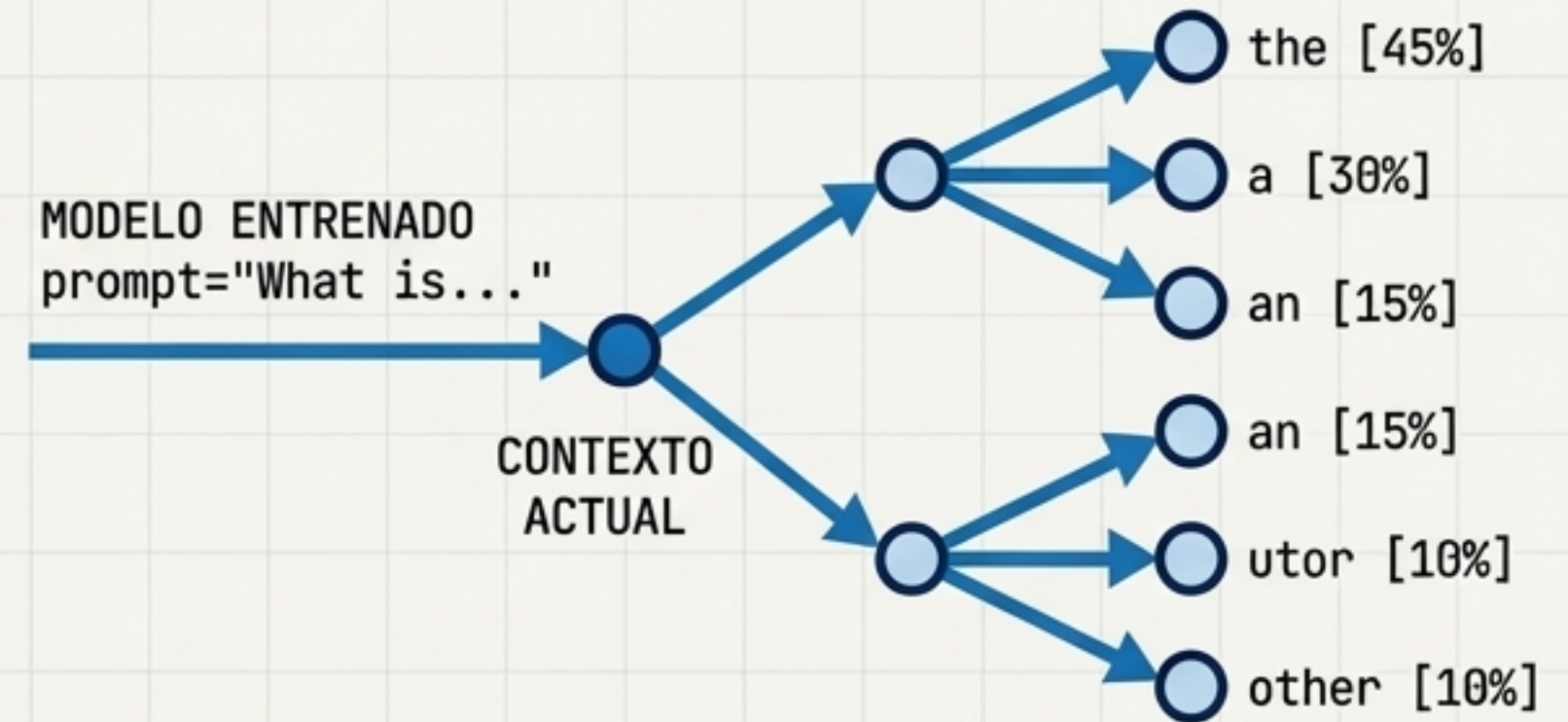
LLM (Large Language Model) - Entidad matemática diseñada para procesar y predecir texto/código. No razona, predice basándose en probabilidades estadísticas.

Fase 1: Entrenamiento



Ingesta masiva de datos ->
Aprendizaje de patrones.

Fase 2: Predicción/Inferencia



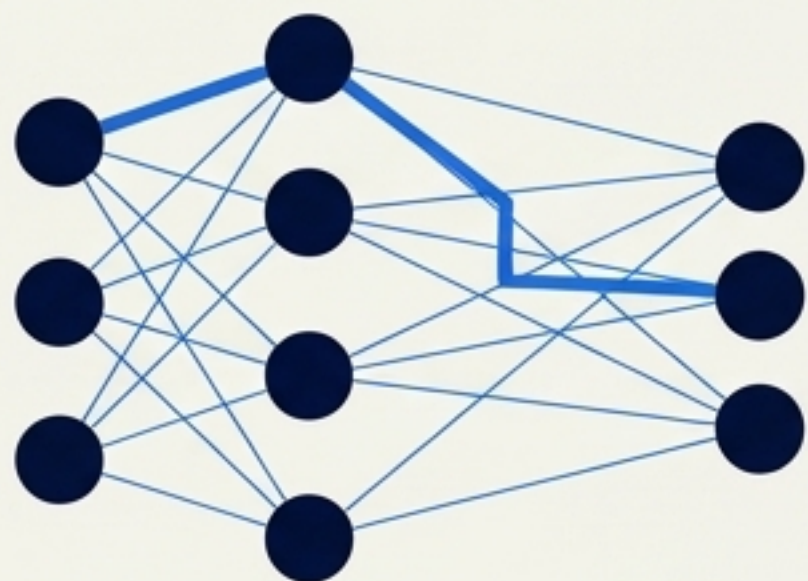
Cálculo estadístico de la siguiente
secuencia de información.

Parámetros: La Resolución del Pensamiento

entrada -> multiplicaciones por parámetros -> resultado

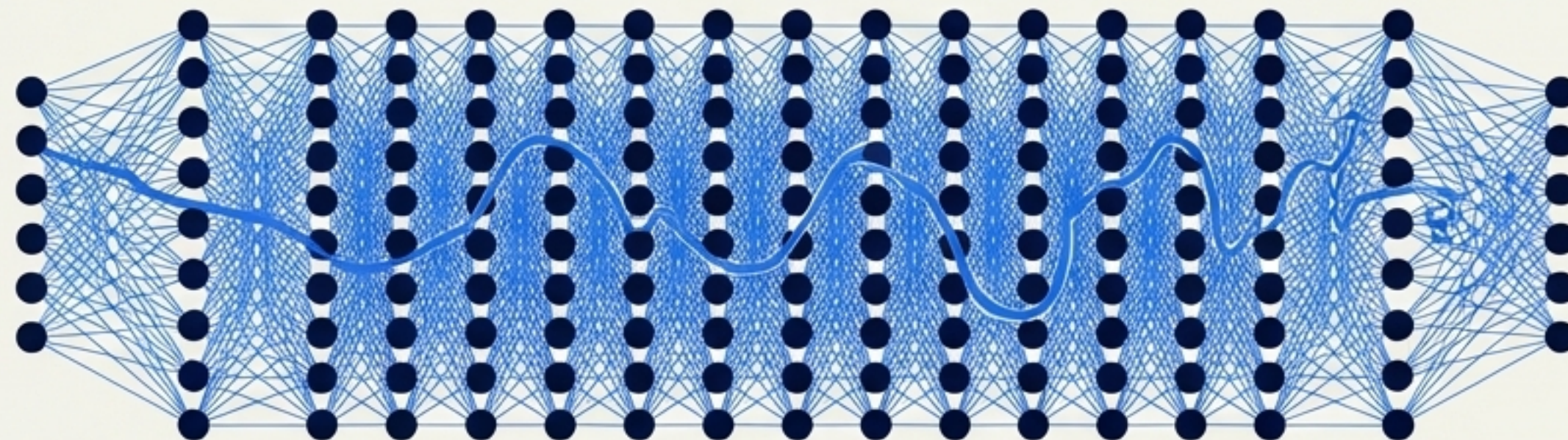
Los parámetros son las conexiones neuronales del modelo.
Almacenan el conocimiento abstracto aprendido.

Menor resolución para
conceptos simples.



1B parameter model

Mayor número de parámetros = mayor finura para representar
conceptos abstractos y arquitectura de código.



70B parameter model

Tokens: Las Piezas de Lego del Modelo

Explí	came	qué	es	un	token	en	un	modelo	de	IA
[435]	[21]	[908]	[712]	[85]	[1204]	[643]	[85]	[3210]	[54]	[160]

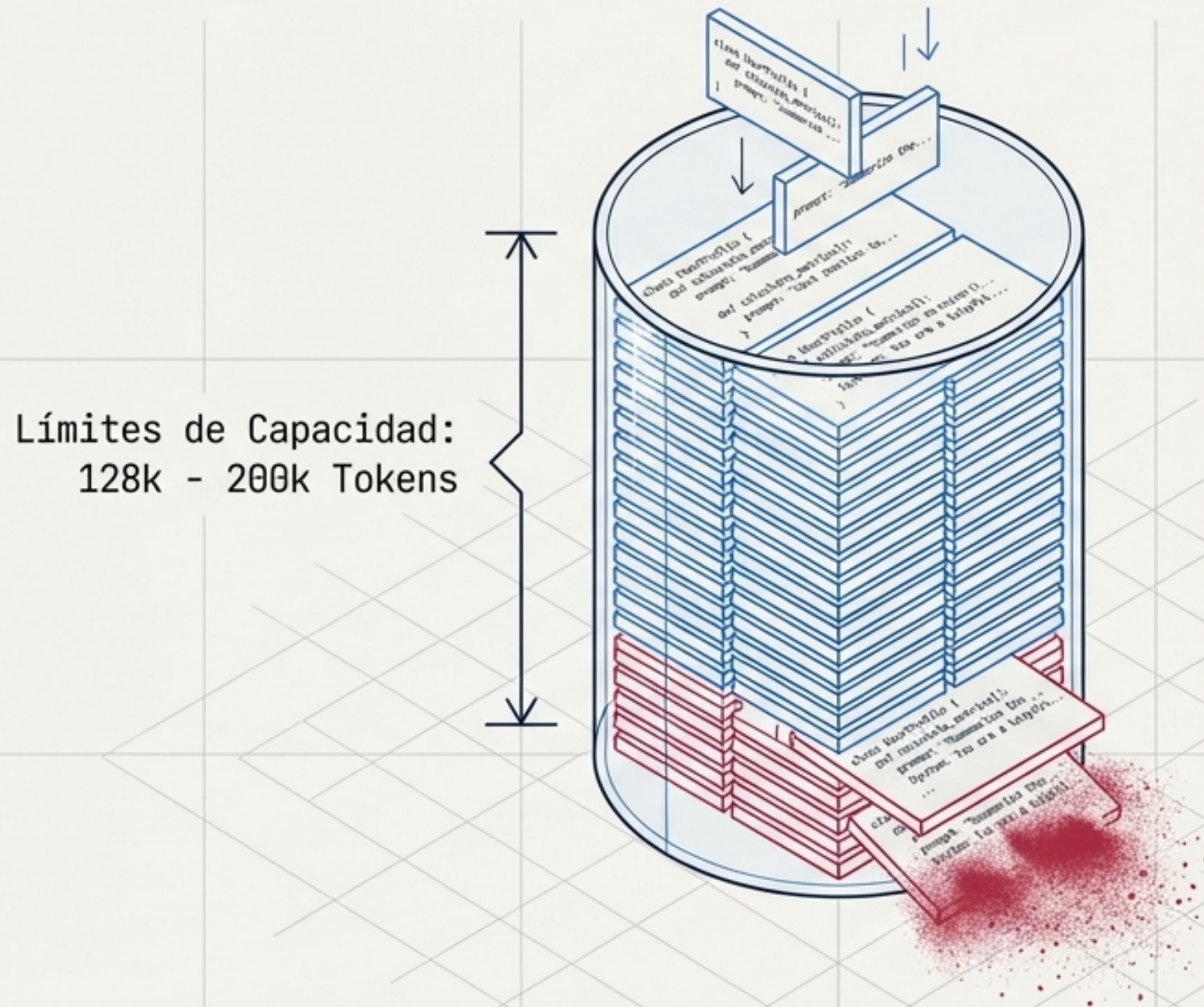
Los LLMs no leen lenguaje humano; procesan fragmentos matemáticos. Esta es la unidad de medida para el procesamiento y el coste computacional.

Lógica de Costes API:

Entrada: \$X / millón de tokens

Salida: \$Y / millón de tokens

La Ventana de Contexto: Gestión de Memoria a Corto Plazo



El Límite Crítico:

Si el proyecto excede el límite de tokens, el modelo olvida los requisitos iniciales. Este desbordamiento es la causa principal de las alucinaciones (cuando la IA inventa funciones o ignora reglas fundamentales).

Telemetría del Modelo: Métricas de Producción

Latencia (TTFT)



120 ms

Tiempo desde el Enter hasta el primer token generado. Crítico para la experiencia de usuario.

Geist

Alucinaciones



El mayor riesgo. Ocurre cuando el modelo inventa información por falta de contexto o conocimiento.

ALERTA DE RIESGO

Inteligencia vs. Velocidad



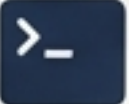
Rápido

Razonamiento

Modelos convencionales (ejecución rápida) vs. Modelos de razonamiento (pausan, planifican y verifican lógicamente antes de responder).

Modo Actual: Razonamiento
(Mayor precisión, menor velocidad).

Arquitectura de Despliegue: Ecosistema Local vs. Cloud

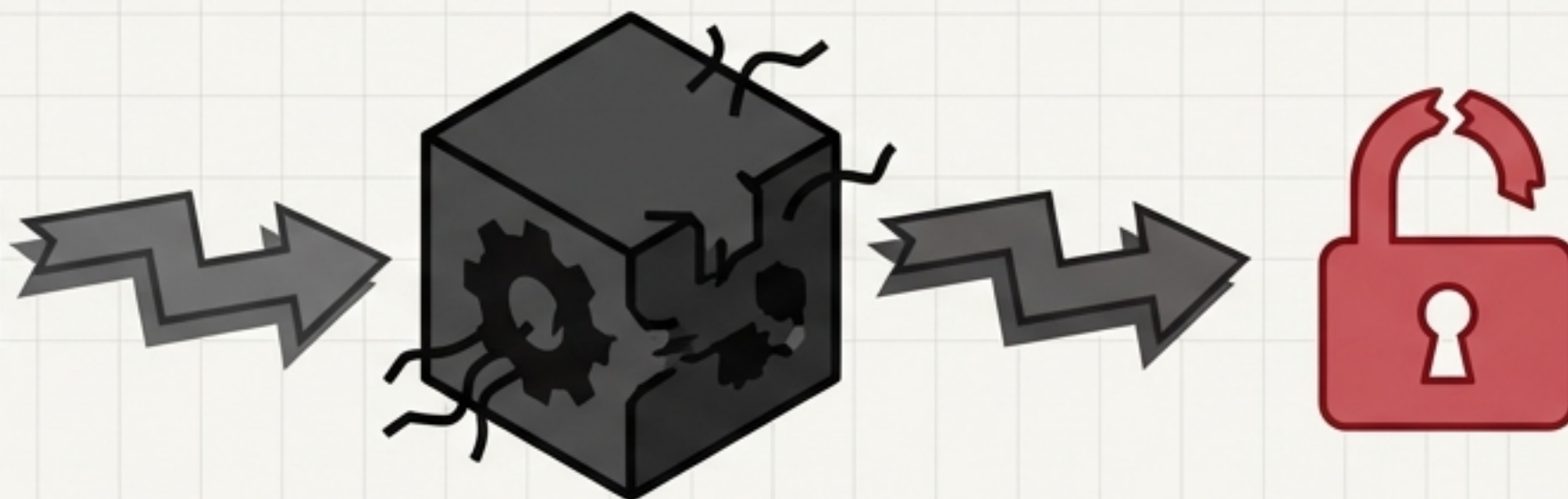
 Local / Open-Source	 Cloud / Privativo
• Herramientas: LM Studio, Ollama.	• Herramientas: APIs REST (OpenAI, Anthropic, Google).
• Privacidad: Absoluta (Soberanía de datos, ideal para código propietario). • Coste: Cero (Ejecución en hardware propio).	• Privacidad: Compartida (Riesgo corporativo al enviar datos a terceros). • Capacidad: Potencia de vanguardia, ventanas masivas.
• Modelos: Llama, Qwen.	• Modelos: GPT-5.4, Claude Sonnet 4.6, Gemini.

La decisión depende del caso de uso: Lógica de negocio core -> Local. Tareas genéricas o Análisis masivo -> Cloud.

Ingeniería de Prompts: Garbage In, Garbage Out

The Bad Prompt

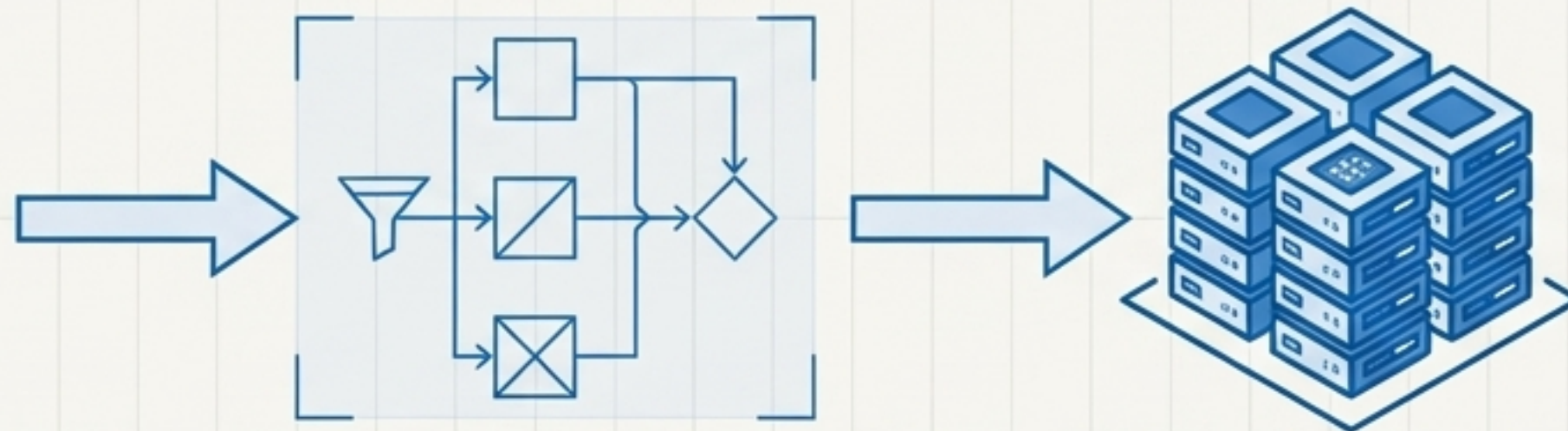
Haz un código para un login en Python.



Código genérico, sin arquitectura, contraseñas en texto plano. Falla en integración.

The Architect's Prompt

[Instrucción estructurada con contexto, rol y reglas]



Microservicio seguro, documentado, adaptado al stack del proyecto.

La calidad del software generado es un reflejo matemático exacto de la arquitectura de tu instrucción.

Anatomía Estructural del Prompt de Alto Valor

Rol: ¿Quién asume la ejecución? Define experiencia y especialidad.

Contexto: ¿Dónde estamos? Stack tecnológico, problema general del ecosistema.

```
<prompt>
<role>
  <!-- Definición del Rol -->
  Expert Backend Engineer with 10+ years experience in high-throughput systems.
  Specialization in secure microservices architecture.
</role>

<context>
  <!-- Contexto del Problema -->
  Working within a Kubernetes environment with a Python/FastAPI stack. Addressing
  scalability bottlenecks in the user authentication service for a FinTech platform.
</context>

<task>
  <!-- Tarea Especifica -->
  Implement a robust, production-ready JWT authentication endpoint in FastAPI. The
  implementation must handle refresh tokens, rate limiting, and secure password hashing
  (Argon2). Do not provide a full system design.
</task>

<constraints>
  <!-- Límites y Reglas -->
  Adhere strictly to OWASP security guidelines. Follow PEP 8 coding standards. All
  database interactions must be asynchronous. Max response time <50ms.
</constraints>

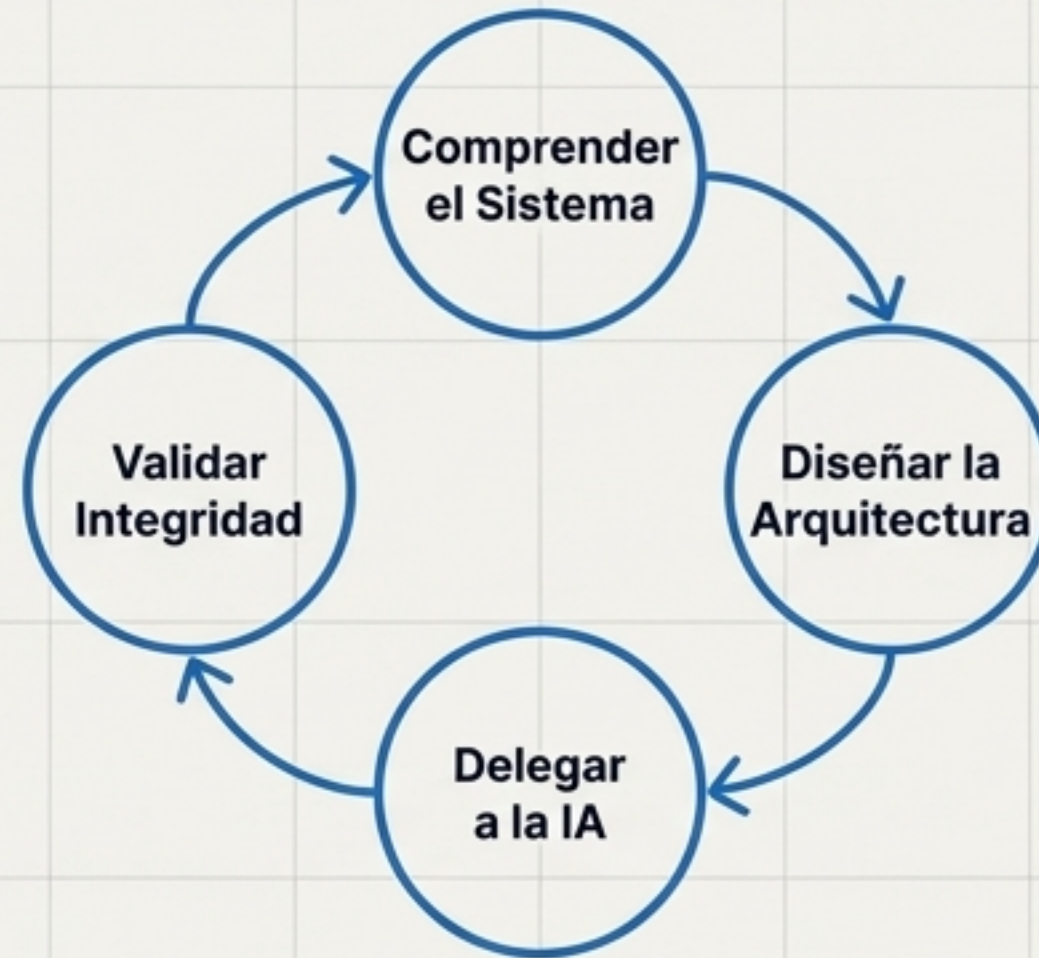
<output_format>
  <!-- Formato de Entrega -->
  Return ONLY the documented code block for the endpoint and necessary dependencies
  in Python. Output as a single, copy-pasteable JSON object with file names as keys
  and code as values. No extra explanations or markdown text outside the JSON block.
</output_format>
</prompt>
```

Tarea Exacta: ¿Qué necesitas? Especificidad extrema, no pedir un sistema abstracto.

Restricciones: ¿Cuáles son los límites? Estándares de seguridad, convenciones de código.

Formato de Salida: ¿Cómo se entrega? JSON, bloque de código documentado, sin explicaciones extra.

El Futuro Pertenece a los Orquestadores



El código es el control.

La programación no muere, evoluciona a un lenguaje de alto nivel.



Los fundamentos mandan.

Sin ingeniería clásica, la IA produce deuda técnica a escala.



Tu nuevo rol.

Eres el piloto, la IA es el motor de ejecución.